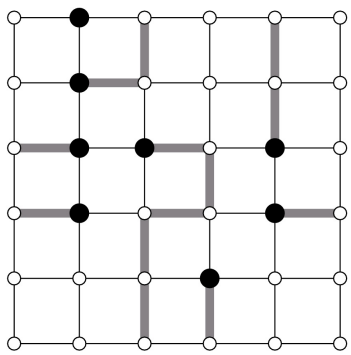
Theory of Computation
Algorithms and Complexity II
Practical 2 (model answers)

Lecturer: Dr. George Mertzios

Model solutions for all exercise sets will be available from duo shortly after the practicals. You should work through them during the following week and make sure that you have understood everything.

Question 1

A building is modeled as a grid. People stand on m points of the grid, and all these people have to be able to leave the building in case of emergency (e.g. fire). Therefore they have to be able to reach a vertex on the border of the grid through some path. It is not allowed that two emergency exit-paths share any vertex. The following picture illustrates an example of such an emergency exit plan:



Model this emergency exit problem as a maximum flow problem.

Answer:

- Take the grid, where all edges are undirected.
- We add a source s. Every vertex with some person on it, becomes connected to s.
- We add a sink t. Every vertex on the boundary of the grid becomes connected to t.
- Every vertex except s, t receives capacity 1.

This maximum flow problem has a solution $\Leftrightarrow$ in the resulting flow network we have a flow that is equal to the number of people who stand on the grid.

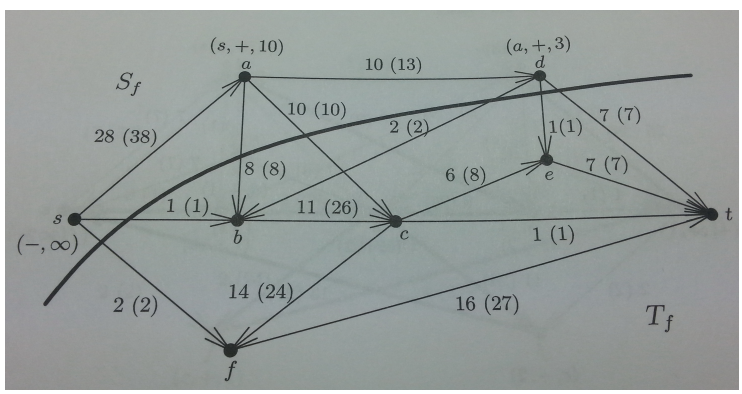
Question 2

Let $N = (G, c, s, t)$ be a flow network and assume that N admits flows of value $\neq 0$. Show that there exists at leats one edge e in N whose removal decreases the value of a maximum flow in N. An edge e is called *most vital* if the removal of e decreases the maximum flow as much as possible. Is an edge of maximum capacity in a minimal cut necessarily most vital?

Answer:

Let W be the maximum value of a flow in the network N and let (S, T) be a minima cut; by hypothesis, $c(S, T) = W \neq 0$. If we remove an edge $e = (u, v)$ from G (where $u \in S$ and $v \in T$ and $c(e) \neq 0$), the capacity $c(S, T)$ and hence the value of a maximum flow is decreased by $c(e)$. This suggests to choose e as an edge of maximum capacity in a minimal cut.

However, these edges do not have to be the most vital ones, as shown on the example of the network in the figure: here the edge (s, a) is clearly the most vital but it is not contained in a minimal cut.



Question 3

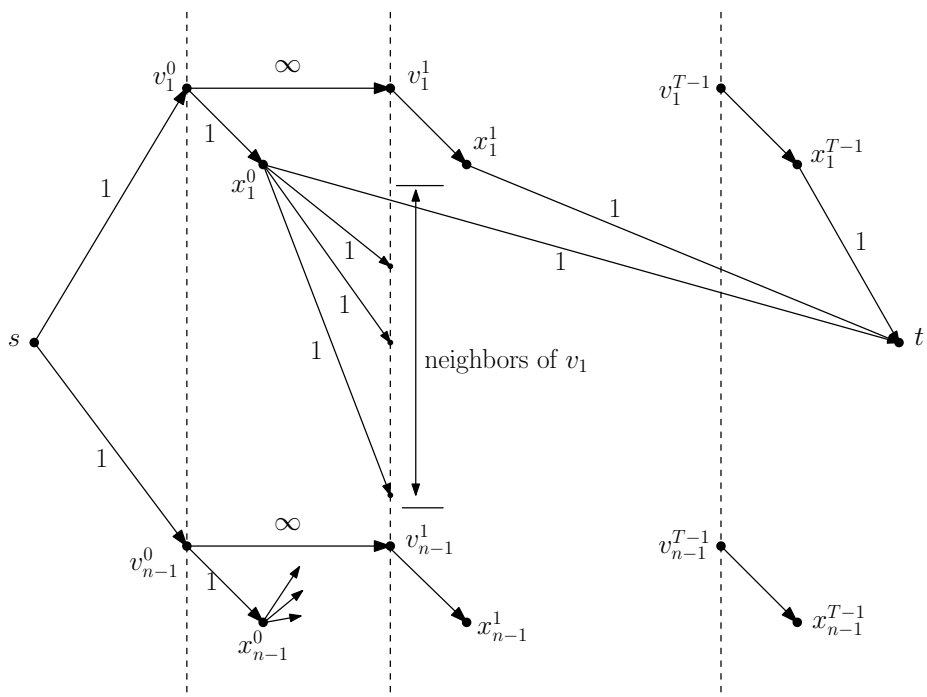
Within a sensor network, sensors receive data that are later collected at a central node of the network. We now want to check whether these data can be collected at the central node within T steps.

Formally we consider the following problem: Let $G = (V, E)$ be a directed graph with n vertices and m edges and let $t \in V$ be a sink (corresponding to the "central node" of the network). Every vertex $v \in V \setminus \{t\}$ has a data packet p_v which has to be transmitted to the sink t. To do that, the packet p_v may have to be forwarded through several other vertices of the graph. We assume that there exists a central clock on the network, which synchronizes all activities of the vertices. At time point t every vertex can send a data packet through an out-going edge. This transmission of the data packet over one edge takes exactly 1 unit of time to be performed (e.g. 1ms). Simultaneously, at any time, every vertex can receive data packets from an arbitrary number of incoming edges.

Model the following decision problem as a maximum flow problem: "can we send the data packets through the network such that all packets arrive at sink t within T time units?"

(Hint: construct a flow network with $2T \cdot (n-1) + 2$ vertices.)

Answer:



Denote the n vertices of the graph by $V = \{v_0 = t, v_1 \cdots v_{n-1}\}$. For every time step $0, 1, \ldots, T-1$ we copy all vertices from v_1 to v_{n-1} (i.e. all vertices without the sink $v_0 = t$) next to each other (in "slices", cf. vertically in the figure). Furthermore we add one copy of the sink vertex t and an additional "source" vertex s (see figure).

First we add one directed edge from the source vertex s to each of the vertices $v_1^0, v_2^0, \ldots, v_{n-1}^0$ (see figure). Furthermore, for every new vertex v_i^j, where $1 \leq j \leq T-1$ (i.e. the copy of vertex v_i at time step j) we add a directed edge with capacity ∞ to vertex v_i^{j+1} (i.e. to the copy of v_i at the next time step $j+1$). This infinite-capacity edge from v_i^j to v_i^{j+1} represents the fact that v_i can keep all its packets during time if necessary.

Now we add for every vertex v_i^j a "forwarding vertex" x_i^j, and we add an edge from v_i^j to x_i^j with capacity 1 (see figure). Then, for every out-going edge $(v_i, v_k) \in E$ in the original graph G, we add an edge from the forwarding vertex x_i^j to vertex v_k^{j+1} (i.e. to the copy of v_k at the next time step $j+1$). The addition of the forwarding vertex x_i^j, together with all these unit-capacity

directed edges (see figure) represent the fact that every original vertex v_i is allowed to transmit at most one packet per time to any other vertex v_k. During this construction, if $(v_i, t) \in E$ (i.e. if there is an out-going edge from v_i to t in the original graph G), then we also add a directed edge from the forwarding vertex x_i^j to the sink t (recall that there is only one copy of t in the flow network).

Note that, by the above construction of the flow network, all directed edges have a direction from left to right (see figure); thus, any directed path in this flow network shows the route of a packet from vertex to vertex as time passes. Furthermore note that, at the last layer of vertices (i.e. $v_1^{T-1}, v_2^{T-1}, \ldots, v_{n-1}^{T-1}$) we also have the corresponding forwarding vertices $x_1^{T-1}, x_2^{T-1}, \ldots, x_{n-1}^{T-1}$. However, each of these forwarding vertices x_i^{T-1} either has no outgoing edges, or it has exactly one out-going edge to the sink t. In the above example, x_1^{T-1} has an out-going edge to t, while x_{n-1}^{T-1} has no out-going edge at all.

Finally, it can be easily verified that, in the constructed flow network, there exists a flow from the source s to the sink t of size $n-1$ if and only if the initial data packet problem in the sensor network G is solvable within T steps. This flow network that we constructed has $2T \cdot (n-1) + 2$ vertices (this can be verified by simple counting of the vertices that we constructed).

Question 4

Consider the following variants of Vertex-Cover:

- DVC – Decision variant: "Does there exist a Vertex Cover of cardinality k?"
- OVC – Optimization variant: "'Which is the cardinality of a minimum Vertex-Cover?"
- MinVC – Construction of the sulution: "Find a smallest Vertex Cover and output it."

Prove that all these variants can be reduced to each other in polynomial time (the part DVC $\leq_p$ OVC $\leq_p$ MinVC is trivial).

(a) Assume that you have an algorithm for DVC with running time $poly(n)$. Give a polynomial-time algorithm for OVC.

(b) Assume that you have an algorithm for OVC with running time $poly(n)$. Give a polynomial-time algorithm for MinVC.

Show the correctness and justify why the constructed algorithms have polynomial running time.

Answer:

(a) $OVC \leq_p DVC$

- binary search over k: Does there exist a Vertex Cover with cardinality k?
  - yes $\Rightarrow$ consider smaller and smaller

- no $\Rightarrow$ consider also larger k.

(b) $MinVC \leq_p OVC$

- Find with OVC the cardinality k of the smallest Vertex Cover in G.
- Remove $v \in V$ from G and compute the cardinality k' of VERTEX COVER in the remaining graph $G - v$. The intuition is the following: we "pretend" that v is included in the Vertex Cover and that all edges incident to v do not have to be covered any more, and thus all the remaining edges of $G - v$ need to be covered with $k - 1$ vertices.

  We distinguish two cases:

  - $k' = k$: in this case, we can not cover the edges of $G - v$ with $k - 1$ vertices, and thus we can not cover the edges of G with k vertices. Therefore $v \notin MinVC$, and thus we put vertex v back in the graph (i.e. we do not include v in the Vertex Cover).
  - $k' < k$: in this case $\Rightarrow v \in MinVC$ (since the edges of $G - v$ can be covered with $k - 1$ vertices). We search further in the graph $G - v$ a Vertex Cover of size $k - 1$.

  Repeat this for every vertex $v \in V$.